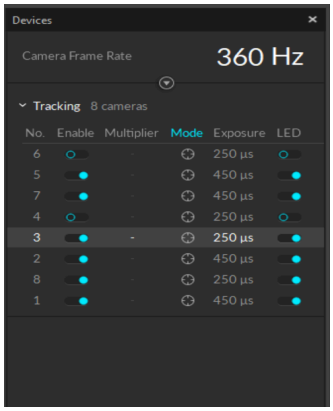
About

- OptiTrack (also referred as MoCap for Motion Capture system) uses an array of cameras to track the 6DoF pose of reflective markers (the shiny silver balls)
- It uses a software "Motive" to interface with the cameras that can only work on Windows and is set up on the computer attached with the OptiTrack system. The license information for the software can be found stuck on the monitor associated with the OptiTrack (Usually you don't have to deal with the licensing).
- The cameras can be left on for many days so at the end of the day you may not have to shut it all down and you can just leave for the day. (That's not energy efficient and would hamper the device life but so far folks have been doing that).

Getting started

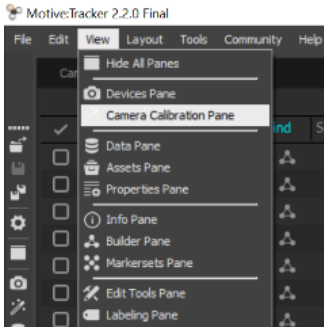
1. **Orient Cameras -** Go around and orient the cameras to point at the region of interest (You have to calibrate the cameras every time you move them).
   In **Motive**, the **Devices** pane shows all the cameras and you can turn them on/off (the camera that is too much occluded should be kept off). The camera number is mentioned below the camera.

2. **Set Frame rate -** Frame rate can also be set here (Devices pane)



3. **Mask (filter out) fringe detections that we don't want to track using Masks** - Clear any existing Masking in Camera Preview using **Camera Calibration > Calibration > Mask**. Set new masks manually or automatically. You set it manually using the Draw Circular Mask icon under Camera Preview. You set it automatically under Camera Calibration > Calibration > Mask. Make sure to not have any moving things in the scene during this step. For manuals, you have to draw red circles over the detections. You can see the masks in the camera preview

If the Camera Calibration Pane is not visible, go to **View > Camera Calibration** Pane.



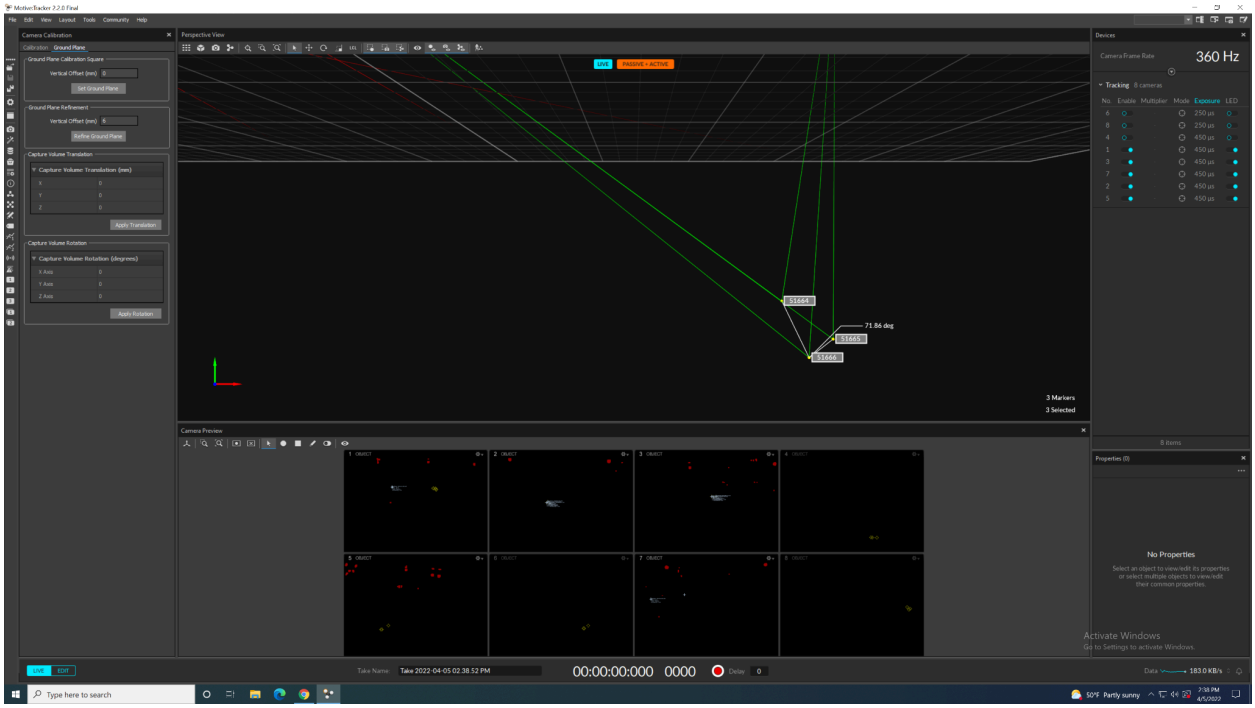
4. **Wanding** - Now you have to ‘ask’ the cameras to see each other. Select all the cameras that you are interested in using (Using Devices Pane). Click Start Wanding (Under Camera Calibration Pane).

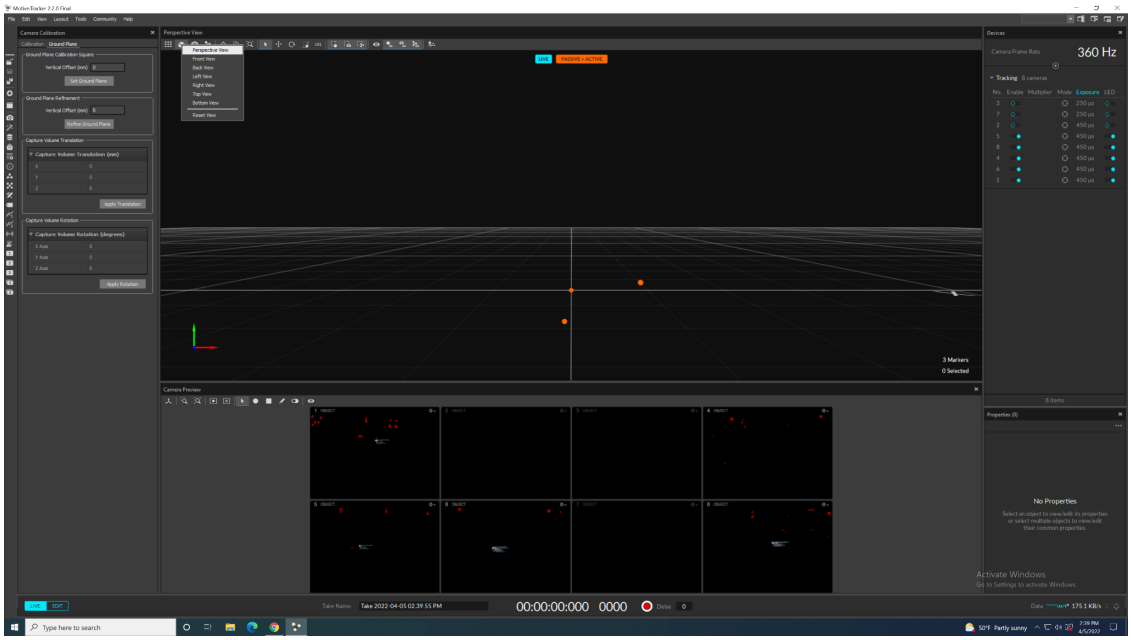
5. **Calibration** -
   Now you have to calibrate using a **T shaped** wand (OptiWand) which has markers placed on it.

Make sure to use either the **A = 500mm** setting **OR** the **B = 250mm** setting. That just means that make sure that the 3 markers are all on the places where it says **A** (or all 3 markers on the places it says B).
Select the relevant tool under **Camera Calibration > Calibration Type > OptiWand**. For **A=500mm**, you have to select **CW-500 (500mm)**.
After that you *slowly* and *smoothly swing* the wand to cover most of the scene. Make the motions smooth and observe the path on the Camera Preview.
Deep blue light (they would be green if they are selected) on the cameras mean that they have been calibrated.

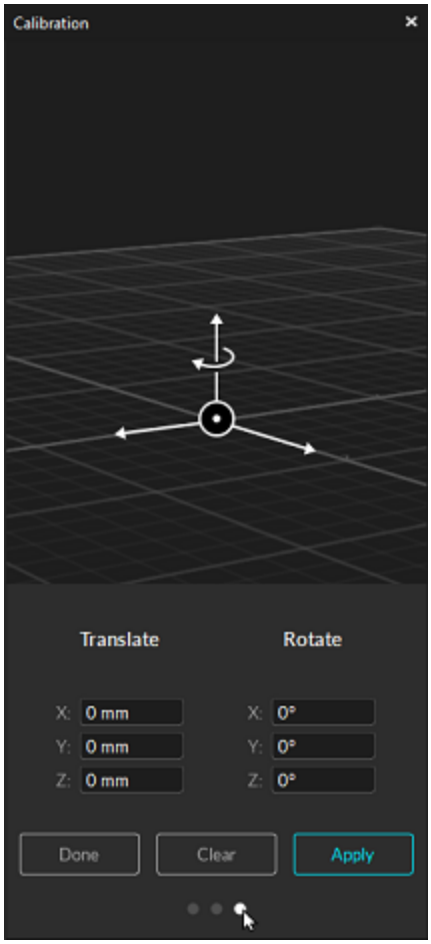
6. **Setting up ground plane and origin -**

You use the above triangular device to set up the ground plane and your origin.
Try to locate the 3 markers on the perspective (or any other type of main view) view. Once done, select all 3 and go under **Camera Calibration > Ground Plane > Set Ground Plane** (change the origin if you need)



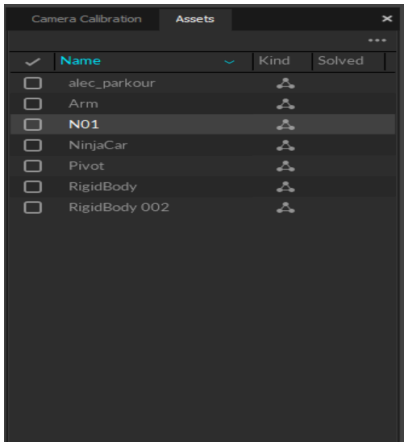


The longer axis is the Z axis and the shorter is the X axis. To match the coordinate system generally used by Sawyer in RVIZ, orient the X-axis facing outward (for Sawyer this meas out from the mounting base blac screws. The Z-axis should be pointing to the “right” relative to Sawyer’s perspective. Rotate the calibration origin positive (+) 90 degrees about the X axis to align the coordinate axes:



Tada, you are finished with the calibration and you should see something like this

7. **Form the rigid body** - This step will help you track a single rigid body. Make sure that the markers are placed unevenly on the rigid body. Locate the markers on the **Perspective View** and select them. **Right Click** and form a rigid body out of them. Make sure to name them appropriately.
   Select the desired rigid body under **Assets Pane** (Helps you select the rigid bodies that you are interested in tracking). Try to not remove other folks' rigid bodies that are previously saved.



8. **Streaming Data** -

   In the **View > Info** Pane, you can see the total numbers of markers being tracked. You should consistently be able to track all the markers on your rigid body.

   Go to **View > Data Streaming** Pane to access this.

   There are ***2 ways*** to send data over wifi (Make sure that the receiver device is on the same wifi network. At the time of writing this tutorial that was **IRLab**).

   a. **VRPN** - This is the *recommended* way. The VRPN based method is based on TCP/IP protocol -
      i. *Pros* - Use this if you want all the data packets to arrive.
      ii. *Cons* -
         1. Requires more installation (3rd party libraries other than python libraries)
         2. Streams just rigid body data and not each marker (This may have changed – yet to verify).
         3. Requires individual clients for each rigid body.
      iii. *How to enable* -
         Under the **Data Streaming** pane, set **Transmission Type** as **Unicast**.
         Set **Broadcast VRPN** as **On**.
         Note the **Local Interface** value (192.168.50.10 for the larger MoCap)

   b. **Natnet** - It uses UDP protocol.
      i. *Pros* - Doesn't have the above mentioned cons.
      ii. *Cons* - Some packets may be lost.

iii. *How to enable* -
Under the **Data Streaming** pane, set **Transmission Type** as **Multicast**.
Note the **Multicast Interface** value (we would reference this as the `client_ip` in the next section).
Example NatNetClient C++

9. **Consuming Data** -
If you are using Macbook M1 pro, look at this thread. If that didn't work for you, Natnet is probably the best option for you.

a. **VRPN** -
Installation: https://gist.github.com/hsharrison/24cbe284bd50973052ee
https://github.com/vrpn/vrpn/blob/master/python_vrpn/README
Sarah's guide:
https://github.com/sarahaguasvivas/sarahaguasvivas.github.io/blob/master/lab_notes/vrpn_client.md

Git clone the vrpn repo,

```
cd vrpn
tar xzvf python_vrpn.tar.gz
cd python_vrpn
edit Makefile.python and set the environment according to your setup
make vrpn-python
(as root) make install-vrpn-python
```

if this doesn't work, try `make install vrpn-python` for the last step.

Example code for client:

```
import vrpn
import numpy as np

class VRPNclient:
    """
        This client has only been tested in Python3.X, 2.7
    """
    def callback(self, userdata, data):
        self.tracked = True
        self.data_read = {userdata: data}

    def __init__(self, tracker_name, hostID):
        self.tracker_name = tracker_name
        self.hostID= hostID
        self.tracked = False
```

```
        self.data_read = None

        self.tracker = vrpn.receiver.Tracker(tracker_name + "@" + hostID)
        self.tracker.register_change_handler(self.tracker_name, self.callback,
"position")
        self.info = []

    def sample_data(self):
        self.tracker.mainloop()

    def get_observation(self):
        while not self.tracked:
            self.sample_data()
        self.info = []
        self.info += list(self.data_read[self.tracker_name]['position'])
        q = list(self.data_read[self.tracker_name]['quaternion'])
        # if you want to explore the variable above, just dir() to see the keys
        self.info += q
        self.tracked = False
        return self.info

if __name__=='__main__':
    import time
    C = VRPNclient("DHead", "tcp://192.168.50.24:3883")
    while True:
        start = time.time()
        print("head: ", C.get_observation()) # collect a single observation
        elapsed = time.time() - start
        print("vrpn elapsed: ", 1./elapsed, " Hz")
```

b. Natnet -

Tutorial and sample code - https://github.com/ratcave/natnetclient (Doesn't work on Ubuntu 20.04, python 3.8, Motive 2.2)

For both large and small optitrack

```
client_ip = 239.255.42.99
data_port=1511, comm_port=1510
```

c. **BEST WAY** - **ROS VRPN**

Installing the ROS package -
`sudo apt-get install ros-noetic-vrpn-client-ros`

Modify this launch file and add the **Local Interface** (IP address of the MoCap system which is **`192.168.50.10`** for the bigger MoCap and **`192.168.50.24`** for the smaller.) value in place of `localhost`.

You can now subscribe to the rostopic
`/vrpn_client_node/[Rigid Body Name]/pose`

(if the topic doesn't exist after successful connection, you might have to move the rigid body at least once for it to register.)

Additionally, you can set a `tf` to obtain the pose w.r.t any coordinate system.

d. **Another ROS based way (based on Natnet)** -
https://tuw-cpsg.github.io/tutorials/optitrack-and-ros/

10. Other resources -
https://v22.wiki.optitrack.com/index.php?title=Skeleton_Tracking